AGENTIC-AI-ENGINEERING · MODULE 03

Advanced Techniques

From prototype to production, and what comes next

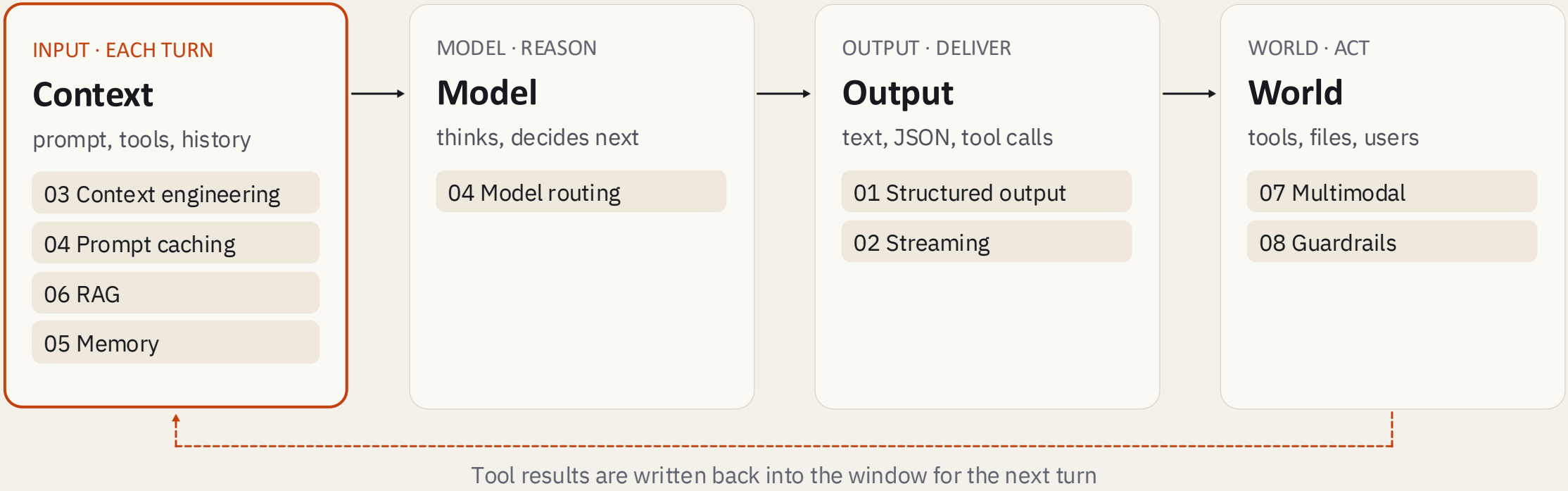


Structured output · Streaming · Context engineering · Cost · RAG · Memory · Multimodal · Guardrails

[Ziqing Shi · October 2026

THE CORE IDEA

An agent lives inside its context window



All eight tutorials answer one question: **each turn, what goes in, what comes out, and what is allowed?**

ROADMAP

Four acts, each topic from today to the frontier

I

Interfaces

How code and people receive the output

01 Structured output

02 Streaming

II

Context

What goes into the window each turn

03 Context engineering

04 Cost optimization

06 RAG

05 Memory

III

World

What the agent can see and do

07 Multimodal

08 Guardrails

IV

Future

What this code turns into

Platformization

Five trends

Thinner harnesses

Two slides per topic: Today = the tutorial's code and approach · Frontier = 2025–2026 platform progress and research

ACT I

Interfaces

How do code and people receive the model’s output?

01 Structured output · 02 Streaming

01 · STRUCTURED OUTPUT · TODAY

From text to types: a reliability ladder

01_structured_output_anthropic.py · excerpt

```
# 2) Native constrained decoding → Pydantic
resp = client.beta.messages.parse(
    output_config={"format": TicketClassification},
    messages=[...])
result = resp.parsed_output
# 3) Validate + retry: rules a schema can't express
class TicketAnalysis(BaseModel):
    requires_escalation: bool
    escalation_reason: str | None = None
    @model_validator(mode="after")
    def check(self):
        if self.requires_escalation \
          and not self.escalation_reason:
            raise ValueError("reason required")
```

1 **tool_use as output**
Force one tool call; its input is the result

2 **Native constrained decoding**
output_config: decoding can only emit valid JSON

3 **Validate + retry**
Feed validation errors back; cap at 2–3 retries

4 **Batch extraction**
3–10 items per call; beyond that, parallelize

Schemas guarantee structure. Validators guarantee business rules. You need both.

▷ uv run --directory 03-advanced-techniques/01-structured-output python 01_structured_output_anthropic.py

01 · STRUCTURED OUTPUT · FRONTIER

Constrained decoding is the default contract

GA · 2026-01

400

Forced tool_choice retires

Structured outputs went GA: output_config.format shapes the answer, strict: true shapes tool inputs. On Opus 5.5 and Fable 5.1, tool_choice any or tool now returns 400 — technique 1 is done.

Research · 2025–2026

+10pp

Think first, then constrain

Constraining too early truncates reasoning. CRANE (ICML 2025): up to +10pp on GSM-Symbolic and FOLIO; Thinking Before Constraining: up to +27%. Claude's grammar applies to the final answer, not to thinking.

Engineering reality

24h

Validation isn't going away

Schemas can't express minimum, maxLength or recursion; compiled grammars are cached for 24 hours, and a request allows at most 20 strict tools. Ranges and cross-field rules still need technique 3.

Migrate tool_use trick → output_config.format + strict tools; keep validation and bounded retries.

Sources: Claude docs, Structured outputs and release notes (2026) · CRANE, ICML 2025 · arXiv 2601.07525

02 · STREAMING · TODAY

Streaming isn’t faster — it’s visible

02_streaming_agent.py · simplified

```
while True:
    with client.messages.stream(tools=TOOLS, ...) as s:
        for ev in s:
            if ev.type == "content_block_start":
                if ev.content_block.type == "tool_use":
                    show_tool(ev.content_block.name)
            elif ev.type == "content_block_delta":
                if ev.delta.type == "text_delta":
                    render(ev.delta.text)
        msg = s.get_final_message()  # usage arrives here
    if msg.stop_reason != "tool_use":
        break  # end_turn / max_tokens
    messages += run_tools(msg)  # execute + append
```

The loop only reads stop_reason

tool_use	Run tools, append results, stream again
end_turn	Done — return the final text
max_tokens	Truncated: continue, or fail loudly

- Total latency is unchanged; perceived latency drops
- One response can interleave text and tool_use blocks
- usage only arrives at the end: get_final_message()
- Stop the Live display in a finally block

▷ uv run --directory 03-advanced-techniques/02-streaming python 02_streaming_agent.py

02 · STREAMING · FRONTIER

From token streams to event streams to UI

STAGE 1 · TOKENS

Tool inputs stream too

Large inputs (whole files, code) no longer wait for complete JSON — the first fragment arrives at once. The price: you must guard against partial or invalid JSON.

`eager_input_streaming`

STAGE 2 · EVENTS

Long tasks stream progress

thinking.display “updates” (beta) streams brief progress between tool calls. Managed Agents keep sessions as append-only event logs, so a crashed harness can recover.

`append-only session log`

STAGE 3 · INTERFACES

Agents stream UI

AG-UI: ~16 standard events into your own app. MCP Apps (Jan 2026, the first official MCP extension) render ui:// UIs in sandboxed iframes in Claude, ChatGPT, VS Code.

`AG-UI · MCP Apps`

Takeaway The question shifts from “how do I print tokens?” to “what should users see while they wait?”

Sources: Claude docs and release notes · Anthropic, Scaling Managed Agents (2026-04) · MCP blog (2026-01) · OpenUI (2026-06)

ACT II

Context

Each turn, what belongs in the window?

03 Context engineering · 04 Cost · 06 RAG · 05 Memory

> Context rot: recall degrades as the window grows. Attention is a finite budget, so aim for “the smallest possible set of high-signal tokens.” — Anthropic

Measure, budget, compress when over

System · fixed | Conversation history · flexible — the part you manage | Reserve = max_tokens

Compression: sliding window + summary

m1 m2 m3 m4 m5 m6 m7 over budget ≈5,000 tok

↓ an LLM summarizes old turns; recent ones stay verbatim

summary ≈200 tok m6 m7 within budget ≈1,500 tok

Tool outputs are the biggest consumers

Strategy	In window	Risk
Naive (as-is)	~1,500 tok	Full after 2–3 calls
Truncate	~150 tok	May cut key facts
Summarize	~200 tok	An extra LLM call per use

```
client.messages.count_tokens(model=MODEL, system=SYSTEM, messages=msgs).input_tokens  # exact pre-flight count
```

▷ uv run --directory 03-advanced-techniques/03-context-engineering python 01_context_engineering_anthropic.py

Compaction as an API, state outside the window

Platform primitive

1 line

ContextManager ≈ a config

clear_tool_uses drops the oldest tool results past a threshold (default 100k; keeps the last 3). Server-side compaction landed in Feb 2026; on-demand compaction (Sep) returns a signed summary block.

Just-in-time context

1–2k

References, loaded on demand

Keep paths and queries in the window; fetch with glob or grep when needed. Hand off across windows via NOTES.md, progress files and a feature-list JSON. Sub-agents digest tens of thousands of tokens, return 1–2k.

MIT · 2025-12

10M+

Context as a variable (RLM)

Recursive Language Models keep the prompt in a REPL variable; the model writes code to slice it and calls itself recursively — inputs ~100× the window. OOLONG-Pairs: from near 0 to 58 F1.

Takeaway The window is a cache, not a database: keep state outside, load only the slice each turn needs.

Sources: Anthropic, Effective context engineering (2025-09) · Claude docs, Context editing and Compaction · arXiv 2512.24601

04 · COST OPTIMIZATION · TODAY

Cache the prefix, route by difficulty

Prompt caching

Call 1 · MISS · write → Then · HIT · read

```
system = [
   {"type": "text", "text": SYSTEM_INSTRUCTIONS},
   {"type": "text", "text": COMPANY_POLICY,
    "cache_control": {"type": "ephemeral"}},
]
# usage.cache_creation_input_tokens → MISS
# usage.cache_read_input_tokens → HIT
```

- Static first, dynamic last: the cache matches prefixes
- 5-minute TTL, refreshed on every hit
- Below the minimum length it’s silently skipped

Model routing

Task → Haiku classifies: easy / hard

easy →
Haiku 4.5
$1 / MTok input

hard →
Sonnet
$3 / MTok input

- The classifier runs on Haiku too; unclear means hard
- Haiku input is 67% cheaper: more easy tasks, more savings
- Misroutes cost quality: add thresholds or a fallback

▷ uv run --directory 03-advanced-techniques/04-cost-optimization python 01_prompt_caching_anthropic.py

04 · COST OPTIMIZATION · FRONTIER

Caching as architecture, routing inside the call

Caching · 2026

0.05×

Append-only context

Cache reads on Opus 5.5 and Sonnet 5.5 cost 0.05× base. Automatic caching needs one top-level cache_control; mid-session system messages and tool changes keep the prefix intact; diagnostics show where it diverged.

In-model routing

effort

One knob for depth

output_config.effort runs from low to max and shapes thinking depth, tool-call count and verbosity together. The docs suggest low for sub-agents and latency-sensitive work, xhigh for 30+ minute coding tasks.

Advisor · 2026-04

41.2%

Small drives, big advises

The executor (Haiku or Sonnet) consults Opus mid-request when stuck and gets back a short plan. BrowseComp: Haiku + Opus advisor 41.2% vs Haiku alone 19.7%, at 85% lower cost per task than Sonnet alone.

Research A 2026 routing survey: self-reported confidence is unreliable — use cascades, verifiers, escalation.

Sources: Claude docs, Prompt caching, Effort and release notes · Claude blog, The advisor strategy (2026-04) · arXiv 2603.04445

06 · RAG · TODAY

Hybrid search: vectors for meaning, BM25 for terms

Docs → Chunk 512/64 → Embed (MiniLM) → Chroma + BM25 → RRF fusion → FlashRank rerank → Claude + citations

rag/retriever.py · simplified

```
def rrf(vector_hits, bm25_hits, k=60):
    """sum of 1/(k + rank): uses ranks, not scores"""
    s = {}
    for hits in (vector_hits, bm25_hits):
        for rank, (c, _) in enumerate(hits):
            s[c.id] = s.get(c.id, 0) + 1/(k + rank + 1)
    return sorted(s, key=s.get, reverse=True)

# 20 candidates each → RRF → cross-encoder → top 5
```

Pipeline vs agentic

	Pipeline	Agentic
Trigger	Every question	Agent decides
Query	User’s words	Agent rewrites
Multi-step	No	Searches again
Best for	Single-turn Q&A	Chat, hard asks

Knowledge base under 200k tokens? Put it all in the prompt — often simpler and more reliable.

▷ uv run --directory 03-advanced-techniques/06-rag-techniques python 02_agentic_rag_anthropic.py

Retrieval: from pipeline to policy

Coding agents

grep

Agentic search wins

Early Claude Code used RAG with a local vector DB; agentic search (grep, glob) proved better and simpler. A May 2026 study across three agent CLIs: grep generally beat vector retrieval, and the harness mattered more.

Retrieval quality

−67%

Contextual chunks still win

Contextual Retrieval adds document-level context to each chunk before embedding: −49% failed retrievals, −67% with reranking. Under 200k tokens, skip RAG: prompt plus caching. 1M context went GA in Mar 2026.

Learning to search

+90.2%

Search is a policy

Anthropic’s multi-agent research system beat single-agent by 90.2%; token usage alone explained 80% of variance. Start wide, then narrow. RL (e.g. Search-R1) trains when and what to search.

Practice — Agentic search as the backbone; a semantic index for concepts and huge corpora. Decide by A/B.

Sources: Boris Cherny (X) · arXiv 2605.15184 · Anthropic, Contextual Retrieval (2024-09), Multi-agent research system (2025-06)

Three tiers; the agent decides what to keep

memory/manager.py · excerpt, simplified

```
MEMORY_TOOLS = [
    {"name": "remember", ...},  # store, with importance
    {"name": "recall", ...},    # search across tiers
    {"name": "forget", ...},    # delete by id
]
# recall: semantic hits ranked by similarity × importance
score = similarity * entry.importance

# session end: LLM extracts key facts → long-term tiers
saved = memory.consolidate(messages, client, MODEL)

# each turn: inject recalls into the system prompt
system = SYSTEM_PROMPT.format(memory_context=...)
```

Tier	Stores	Backend	Search
Working	This session	List	Direct
Episodic	Events	JSON	Keywords
Semantic	Facts, likes	ChromaDB	Vectors

Capture → Store → Recall → Consolidate → Forget

The tutorial's next step: add procedural memory. That's exactly where the frontier is going.

▷ uv run --directory 03-advanced-techniques/05-memory python 01_memory_agent_anthropic.py

Memory as files, consolidated while “asleep”

Storage

Files

Memory you can manage

The memory tool lets Claude read and write files in a memory directory. Managed Agents memory stores (beta, Apr 2026) mount as a filesystem: shared across agents, with scoped permissions, audit logs and rollback.

Offline consolidation

Dreams

Tidy memory while asleep

Dreams (preview, May 2026) reads a store plus 1–100 past sessions: merges duplicates, replaces contradicted entries, surfaces insights, and writes a new store. Letta’s sleep-time compute is the same idea.

Procedural memory

+10.6%

From facts to know-how

Skills (SKILL.md + progressive disclosure) are an open standard, supported by Codex, Gemini CLI, Copilot, Cursor and more. ACE (ICLR 2026) treats context as an evolving playbook: +10.6% agent, +8.6% finance.

But ContinualSkillBench (Aug 2026): plain in-context learning matched explicit skill upkeep (0.605 vs 0.602).

Sources: Claude docs (Memory tool, Managed Agents, Dreams) · Letta (2025-04) · agentskills.io · arXiv 2510.04618, 2608.03874

ACT III

World

When agents start to see, hear and act — where are the boundaries?

07 Multimodal · 08 Guardrails

Three modalities, three providers, one format

VISION

Seeing

Claude

```
{"type": "image",
 "source": {"type": "url", "url": image_url}},
{"type": "text", "text": "Describe this image."}
```

URL or base64; label images when comparing; a 1568px image ≈ 1,600 tokens

IMAGE

Generating

Gemini

```
cfg = types.GenerateContentConfig(
    response_modalities=["TEXT", "IMAGE"])
# image bytes come back as inline_data in parts
```

One model understands and generates; the tutorial uses an experimental gemini-2.0 model

AUDIO

Speech

OpenAI

```
client.audio.speech.create(
    model="tts-1", voice="alloy", input=text)
client.audio.transcriptions.create(
    model="whisper-1", file=f)
```

TTS → audio → Whisper → text: a round trip that checks fidelity

▷ uv run --directory 03-advanced-techniques/07-multimodal python 01_vision_anthropic.py

07 · MULTIMODAL · FRONTIER

From input format to eyes and hands

Computer use · GA 2026-08

85%

From seeing to doing

computer_toolset_20260801 is GA with batched actions and zoom, plus a new browser use tool. Self-reported OSWorld-Verified scores reach 85–86%; the original OSWorld human baseline is 72.36% (not directly comparable).

Perception & generation

2576px

Sharper eyes, better edits

Image input grew from 1568px to 2576px on the long edge (Opus 4.7+). Nano Banana 2 and Gemini Omni Flash add conversational editing and video; the tutorial's gemini-2.0 model is a generation behind.

Voice

<1s

Cascaded or native?

Cascaded (STT → LLM → TTS) exposes transcripts and lets you swap each layer; native speech-to-speech (e.g. OpenAI's Realtime models) is sub-second and more natural, but less transparent.

Next Once a model can click, type and submit, the next question is: what is it allowed to do?

Sources: Claude release notes (2026-04, 2026-08) · OSWorld leaderboard (steel.dev) · Google blog (2026-06) · AssemblyAI (2026-09)

Defense in depth: cheap checks first, LLM last



Layer	Catches	Latency	Cost
Length limit	Many-shot injection, token exhaustion	<1ms	$0
Regex patterns	Known injection phrases	<1ms	$0
PII scan	SSNs, cards, emails (warn only)	<1ms	$0
LLM screen (Haiku)	Novel attacks, subtle manipulation	200–500ms	≈$0.01 / 1K msgs
Output policy check	Violations, leaked internals	200–500ms	≈$0.02 / 1K msgs
Groundedness	Hallucinations, unsupported claims	200–500ms	≈$0.02 / 1K msgs

```
r"ignore\s+(all\s+)?(previous|above|prior)\s+(instructions|rules|prompts)"  # layer 2: regex
passed = risk_level < 2  # layer 4: Haiku scores 0–3; fails OPEN if the classifier errors
```

▷ uv run --directory 03-advanced-techniques/08-guardrails python 01_guardrails_anthropic.py

From filtering content to limiting capability

Research · 2025-10

>90%

Adaptive attacks win

The Attacker Moves Second tested 12 published defenses: adaptive attacks succeeded over 90% of the time against most, and human red-teamers beat all 12. In security, 99% detection is a failing grade.

Meta · 2025-10

2 of 3

Agents Rule of Two

In one session, allow at most two of: untrusted input, sensitive data, changing state or communicating out. All three requires human oversight. Patterns: plan-then-execute, dual LLM, CaMeL.

Anthropic · 2026-05

93%

Bound the blast radius

Users approved 93% of permission prompts, so approval alone fails. Set hard limits in the environment first: sandbox, egress control, no credentials inside. Sandboxing cut prompts by 84%.

New surfaces — Persistent memory poisoning, over-trusted sub-agent output, and the agent’s own identity.

Sources: The Attacker Moves Second (2025-10) · Meta AI (2025-10) · arXiv 2506.08837 · Anthropic, How we contain Claude (2026-05)

ACT IV

Future

When these techniques become platform features, what should engineers focus on?

Platformization · Five trends · Code as context · Thinner harnesses

TREND · PLATFORMIZATION

The code you wrote is becoming a parameter

Tutorial	What you built by hand	Platform primitive (2025–2026)
01 Structured output	Forced tool_use as output	output_config.format · strict: true
03 Context engineering	ContextManager summarization	context_management · compaction
04 Cost	Manual cache_control breakpoints	Automatic caching · 1h TTL · diagnostics
04 Cost	Haiku difficulty router	effort · advisor tool
05 Memory	MemoryManager + consolidate	Memory stores · Dreams
06 RAG	Vector + BM25 + rerank pipeline	Retrieval as a tool: agentic search · web_search
08 Guardrails	Regex + Haiku input screen	Sandboxing · injection probes · auto permissions

So why build it from scratch? Only by knowing the primitives can you debug, compose and evaluate the managed versions — that’s the method of this course.

FIVE TRENDS

Five directions from today to the frontier

1	**Pull, don’t push**	The model fetches context on demand; we stop pre-stuffing it agentic search · tool search (–85% definition tokens) · progressive Skills
2	**Code is context**	Data flows through code; only conclusions enter the window programmatic tool calling · code execution with MCP · RLM · CaMeL
3	**State lives outside**	The window is a cache; the truth lives in logs and files session event logs · memory stores · progress notes · compaction
4	**Agents that learn**	Experience is consolidated offline into reusable knowledge Dreams · sleep-time compute · Skills · ACE — clear direction, unsolved problem
5	**Safety by environment**	Capability limits come before content filtering Rule of Two · sandboxing and egress control · credential isolation

TREND 2 · DEEP DIVE

Code is the new context

Classic tool calling: every intermediate result passes through the window

Model → Tool A → Result in window → Model → Tool B → Result in window → …

Code execution: data flows in the sandbox; the window gets conclusions

Model → Write a script → Sandbox [Tool A → Filter → Tool B → Summarize] → Conclusion only → Model

–98.7%

Code execution with MCP: one workflow, ~150k → 2k tokens

–85%

Tool search: tool definitions ~77k → 8.7k tokens

–37%

Programmatic tool calling: 43,588 → 27,297 tokens on research tasks

100×

RLM: inputs about two orders of magnitude beyond the window

Same idea — Security too: code-then-execute and CaMeL make data flow traceable and enforceable.

Sources: Anthropic, Code execution with MCP; Advanced tool use (2025-11) · arXiv 2512.24601 · DeepMind, CaMeL (2025)

META-TREND

Stronger models, thinner harnesses

> Every harness component encodes an assumption about what the model can’t do — and those assumptions expire.
>
> Paraphrasing Anthropic, Harness design for long-running apps (2026-03)

- Sonnet 4.5 showed “context anxiety” and needed context resets; with Opus 4.5 it mostly vanished, so resets were removed
- After Opus 4.6, the sprint structure was dropped; the evaluator only pays off at the edge of capability
- Managed Agents decouple brain, hands and session: stable interfaces, replaceable parts

METR · 50%-success task length

4.3 months to double

Doubling time since 2023; about 7 months over 2019–2024

≈12h	Claude Opus 4.6 · Feb 2026
≥16h	Mythos Preview · Apr 2026, at the measurement ceiling

Today’s patch is tomorrow’s deletion.

Sources: Anthropic Engineering (2026-03, 2026-04) · METR, Time Horizon 1.1 (2026-01) and 2026 measurements

TAKEAWAYS

Three things to take home

1

Learn the primitives, use the platform

Building from scratch is how you understand managed features. In production, start with platform primitives and add complexity only when evals say so.

2

The window is a cache

Append, don’t rewrite. Keep full state outside the window and load only the slice each turn needs.

3

Environment before model

Bound what the agent can do with sandboxes, permissions and egress control; then steer how it does it with prompts and classifiers.

Q & A

Thank you

github.com/agenticloops-ai/agentic-ai-engineering

The code lives in 03-advanced-techniques/. The next module, 05 Loop Engineering, builds Skills, hooks, sandboxing, MCP, sub-agents and compaction from scratch — the primitives behind today's frontier.

APPENDIX A

Live demo checklist

```
uv run --directory 03-advanced-techniques/<dir> python <script>  # set API keys in .env first (see SETUP.md)
```

#	Demo	Directory	Script	What to show
1	Self-healing	01-structured-output	01_structured_output_anthropic.py	Technique 3: fail → fix
2	Context budget	03-context-engineering	01_context_engineering_anthropic.py	4K budget compresses
3	Cache hit	04-cost-optimization	01_prompt_caching_anthropic.py	2nd call: cache_read
4	Agentic RAG	06-rag-techniques	02_agentic_rag_anthropic.py	Agent rewrites queries
5	Guardrails	08-guardrails	01_guardrails_anthropic.py	Blocked at layer 2

Stretch comparison: after Demo 2, replace ContextManager with a one-line clear_tool_uses config and compare code size and behavior.

APPENDIX B

References

Course and official sources

- agentic-ai-engineering · 03-advanced-techniques (GitHub)
- Claude docs: Structured outputs, Prompt caching, Context editing, Compaction, Effort, Memory, Dreams
- Claude Platform release notes (Jan–Oct 2026)
- Anthropic Engineering: Effective context engineering (2025-09) · Code execution with MCP · Advanced tool use · Effective harnesses for long-running agents (2025-11)
- Harness design for long-running apps (2026-03) · Scaling Managed Agents (2026-04) · How we contain Claude (2026-05) · Claude Code auto mode (2026-03)
- Contextual Retrieval (2024-09) · Multi-agent research system (2025-06) · The advisor strategy (2026-04)

Research and community

- Recursive Language Models · arXiv 2512.24601
- ACE: Agentic Context Engineering · ICLR 2026
- Is Grep All You Need? · arXiv 2605.15184
- ContinualSkillBench · arXiv 2608.03874
- CRANE (ICML 2025) · Thinking Before Constraining · arXiv 2601.07525
- Dynamic Model Routing and Cascading · arXiv 2603.04445
- The Attacker Moves Second (2025-10) · Agents Rule of Two (Meta, 2025-10)
- Design Patterns for Securing LLM Agents · arXiv 2506.08837 · CaMeL
- MCP Apps (2026-01) · State of Generative UI (OpenUI, 2026-06)
- METR Time Horizon 1.1 · OSWorld-Verified leaderboard · Letta sleep-time compute